

OTRO: Oblivious Tokenization Path with Square-Root ORAM

Jonghyun Lee*, Yongqin Wang[†], Rachit Rajat[‡], Daniel Wong[§], Mengyuan Li[¶], and Murali Annavaram*

*Ming Hsieh Dept. of Electrical and Computer Engineering, University of Southern California, {leejongh, annavara}@usc.edu

[†]Roblox, yongqinwang@roblox.com

[‡]NVIDIA, rrajat@nvidia.com

[§]Dept. of Electrical and Computer Engineering, University of California, Riverside, daniel.wong@ucr.edu

[¶]Thomas Lord Dept. of Computer Science, University of Southern California, mli49061@usc.edu

Abstract—The CPU-side large language model (LLM) tokenizer is a critical security gap in LLM serving through a confidential computing stack with CPU and GPU trusted execution environments (TEEs). Tokenizers convert the prompts through table-driven lookups, and the resulting memory access patterns are a powerful source of side-channel leakage. Recent work demonstrates end-to-end recovery of user prompts from tokenizer access pattern on production Intel TDX. However, a drop-in use of the popular tree-based Oblivious RAMs (e.g., PathORAM) to prevent access-pattern leakage introduces $\sim 13\times$ tokenizer slowdown, resulting in 10–58% higher time-to-first-token (TTFT).

In this paper, we present **OTRO**, an efficient, oblivious tokenization path tailored to latency-critical LLM serving. **OTRO** relies on square-root ORAM for fast single-access lookups, but avoids its prohibitive $O(N \log^2 N)$ rebuild cost every $\sqrt{N}$ accesses through three key innovations. First, **OTRO** provides a pool of replicated square-root ORAM instances that utilize the read-only nature of tokenizer table. Second, an epoch-based rotation policy decouples accesses from rebuilds and pads each epoch with dummy accesses to its boundaries, minimizing observable information. Lastly, chunked KV-cache-aware tokenization further overlaps rebuilds with GPU prefill and minimizes the instance count. Implemented as modules in HuggingFace Tokenizers and nano-vLLM, running within a TDX-enabled CVM with an NVIDIA H100 GPU, **OTRO** limits TTFT overhead to at most 4.5%, keeps tokenizer-induced latency under 10% of total TTFT, and adds less than 0.5 GB of memory overhead while reducing the tokenizer’s observable leakage across various model families and sizes.

1. Introduction

Large language models (LLMs) are ubiquitous in everyday tools and services, powering interactive assistants, chatbots, and other conversational AI systems. As LLMs are often used to process highly sensitive inputs, including proprietary documents, personal communications, and security-critical prompts, naively entrusting LLM usage to a cloud service provider (CSP) creates various privacy concerns [1], [2], [3], [4], [5]. Ensuring that user prompts

and AI-generated responses remain unobservable to CSP or other malicious users on CSP (thus, **confidential**) during *cloud-based* LLM inference is paramount for maintaining user trust and meeting legal and ethical data protection standards.

The Emergence of Confidential LLM Serving. To address these concerns, LLM serving is rapidly adopting Trusted Execution Environments (TEEs) [6], [7], [8]. TEEs provide hardware-isolated environments that encrypt memory and restrict address access requests, ensuring that computations inside a confidential virtual machine (CVM) remain inaccessible even to the cloud provider. CVMs protect model weights, KV caches, and user prompts, and can be verified through remote attestation [7], [8], [9]. Building on CVMs, GPU vendors have also introduced GPU TEE support (also called confidential computing) tailored to large-scale confidential LLM inference [10], including NVIDIA Hopper H100/H200 [11] and Blackwell (e.g., B200 [12]).

With this hardware foundation maturing, major technology companies are beginning to assemble complete confidential LLM inference stacks; Microsoft, Meta, and Google are integrating CPU and GPU TEEs into end-to-end frameworks that enforce privacy guarantees [10], [13], [14]. For example, Meta’s recently announced “Private Processing” framework [13] uses confidential computing to deploy LLM-powered features (e.g., summarization of WhatsApp messages [15]) while ensuring that sensitive user data remains invisible to the underlying cloud provider and even Meta itself. Taken together, these efforts signal the emergence of a new baseline: an LLM serving pipeline in which confidentiality is guaranteed by hardware-enforced TEEs. Under this model, *a malicious cloud LLM operator or hypervisor cannot recover the user prompt, intermediate states, or generated responses*, achieving end-to-end privacy for practical large-scale deployments.¹

Tokenizer: A Demonstrated Source of Leakage in Confidential LLM Serving. However, even with CVMs, one component of the LLM serving pipeline remains a source

1. Denial-of-service attacks fall outside the scope of confidential LLM serving, as they degrade availability rather than confidentiality and can be readily observed by end users.

of information leakage: tokenizer. A tokenizer splits the prompt into a sequence of short vocabulary terms (called tokens). Tokenizer’s memory access patterns, if observed by adversaries, can leak the prompts (more details about the adversaries in Section 3.1) [16], [17], [18], [19], [20], [21], [22], [23], [24], [25]. CVMs only protect memory content and the computations on data but they do not protect against address pattern leakage. In this work, we target the byte-pair encoding (BPE)-style sub-word tokenizers, which are widely used in many modern decoder-only LLMs [26], [27], [28], [29]. BPE tokenizers use a vocabulary and merge table, together with hash-based dictionary lookups, to map raw prompt text into a sequence of token identifiers (tokenIDs). The tokenizer relies on a hash map with a randomized seed chosen at initialization of the LLM serving process. Once constructed, the mapping from tokens to vocabulary table entries (hence the memory addresses) remains fixed for the lifetime of the inference engine. A malicious/compromised hypervisor can leverage page faults and cache side channels to correlate observed address traces with lookups under this fixed mapping for a given prompt (more details in Section 3.2). Without additional protection, the address patterns of tokenizer memory accesses can leak the prompt’s content, even though data remains encrypted in TEE memory. This attack is different from token-length based attacks [1], [30], where an adversary infers prompt characteristics from prompt length. In contrast, address pattern leakage enables deterministic prompt reconstruction.

LLM tokenizer is not a hypothetical attack vector. Recent work, TDXRay [31], demonstrates an end-to-end prompt-reconstruction attack on Intel TDX by tracking the tokenizer memory accesses, achieving over 90% reconstruction similarity across different model families. The tokenizer is therefore a high-fidelity prompt-reconstruction channel that survives the TEE protections deployed to prevent it. TDXRay discuss oblivious map as a mitigation candidate but finds the straightforward implementation prohibitive on the tokenizer path (53–369 $\times$ overhead), leaving an efficient oblivious tokenizer an open problem.

Prior works [32], [33], [34] demonstrated side-channel leakage in embedding layers on recommendation systems and LLMs, showing that memory access patterns in embedding table lookups can reveal sensitive inputs. Embedding layers, however, operate on tokenIDs generated by tokenization, whereas the tokenizer directly processes raw user prompts before any model computation occurs. Leakage at the tokenizer stage, therefore, exposes fine-grained user content at the earliest stage in the LLM pipeline. Protecting against tokenizer leakage constitutes a distinct systems problem with different architectural constraints from embedding protection.

1.1. OTRO: Efficient and Oblivious Tokenization Path

In this work, we propose OTRO, an efficient and secure Oblivious Tokenization Path tailored to confidential LLM serving. Building on Oblivious RAM (ORAM), which hides

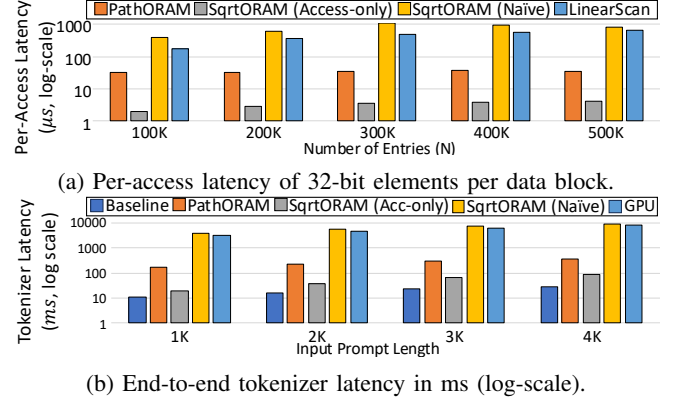


Figure 1: Latency of ORAM and GPU-based tokenizers.

access patterns by making any two logical access sequences of the same length computationally indistinguishable [35], [36], [37], we investigate whether ORAM-based tokenizer protection can be made practical at the system scale. Existing tree-based ORAM, such as PathORAM [36], is a widely adopted baseline for general-purpose ORAM with strong security guarantees. But tree-based ORAMs incur significant overhead for randomizing tokenizer accesses, as we demonstrate later. Tree-based ORAMs organize N entries into a logarithmic-depth hierarchy causing only poly-logarithmic overhead in theory. However, when embedded into tokenizer lookups, each token access must traverse an entire ORAM path and perform complex eviction. In our measurements, this translates into up to 58% higher TTFT relative to a non-oblivious baseline, unacceptable for latency-critical inference.

Naive Defenses Fall Short. Figure 1 quantifies the cost of naive protection. At the per-access level (Figure 1a), Square-root ORAM (SqrtORAM)’s single-block lookup averages only 3.2 μ s, 10 $\times$ faster than PathORAM. When integrated into HuggingFace’s Tokenizers (Figure 1b), PathORAM and naive SqrtORAM incur 13 $\times$ and 323 $\times$ tokenizer slowdowns respectively; the former from full tree path traversals with small lookups per tree node, the latter from rebuild cost dominating the runtime. Even GPU offloading shows a 276 $\times$ slowdown, as BPE’s greedy merge causes severe control-flow divergence that defeats SIMD parallelism. Among all these, SqrtORAM stands out for one reason. Isolating its per-access cost from rebuilds reveals only 2.4 $\times$ overhead over baseline without any protection, which shows that the rebuild, not the oblivious lookup itself, is what makes naive SqrtORAM 323 $\times$ slower. This motivates our key question, *can rebuild cost be removed from the critical path entirely?*

Our *key observation* is that the tokenizer tables are read-only structures during inference. Square-root ORAM (SqrtORAM) is an existing approach that offers near-unprotected-baseline per-access cost with one drawback: SqrtORAM suffers from expensive $O(N \log^2 N)$ rebuilds every $\sqrt{N}$ accesses. OTRO eliminates rebuild overhead by constructing a *pool of replicated read-only SqrtORAM instances* and decoupling accesses from rebuilds via an

epoch-based rotation policy. Each instance, after serving $\sqrt{N}$ accesses (an epoch), goes offline for oblivious rebuild, while subsequent tokenizer requests are routed to a fresh instance. OTRO leaves tokenizer access patterns provably independent of the prompt content under a DRAM-trace adversary, revealing nothing about which vocab or merge entries are touched. Additionally OTRO pads each epoch to its $\sqrt{N}$ boundary with dummy accesses, reducing the observable transcript to input prompt length, which is unavoidable in any ORAM design.

To further reduce the number of instances in a pool and hide rebuilds from the critical path, OTRO introduces *chunked tokenization* which overlaps the prefill of prompt chunks with the rebuild phase via incremental prefill over a shared KV cache. Together, these techniques convert the bursty rebuild cost of SqrtORAM into amortized background work, keeping ORAM-induced stalls off the LLM pipeline’s critical path and yielding an oblivious tokenizer that closely tracks non-oblivious TTFT. Our key contributions are:

- **Cost characterization of oblivious tokenization.** We implement drop-in oblivious defense on the tokenizer. PathORAM and naive SqrtORAM are prohibitive ($13\times$ and $323\times$), but rebuild-free SqrtORAM tracks baseline, which has no protection, by under $2.4\times$. Since the rebuild is the bottleneck, OTRO keeps it off the critical path while retaining SqrtORAM-based protection.
- **Oblivious tokenization architecture.** OTRO introduces a pool of read-only SqrtORAM instances with an epoch-based rotation strategy and access-count padding, leveraging the static nature of tokenizer tables to remove rebuilds from the critical path.
- **Latency-aware ORAM integration.** We integrate OTRO with chunked tokenization to keep rebuilds off the critical path and reduce the memory footprint. OTRO overlaps ORAM maintenance with GPU prefill by chunking the user prompt and using KV-cache-aware incremental decoding. This integration ensures that SqrtORAM rebuilds remain off the LLM pipeline’s critical path and reduces the number of SqrtORAM instances.
- **End-to-end prototype.** We implement and evaluate OTRO in a real-world confidential serving stack. We extend HuggingFace Tokenizers and nano-vllm to support OTRO inside a TDX-enabled CVM with an NVIDIA H100 GPU. Our evaluation shows that OTRO only limits TTFT overhead to at most 4.5% and adds less than 0.5 GB of memory, significantly outperforming PathORAM and naive SqrtORAM baselines.
- **Reducing leakage bound.** We prove that OTRO’s access patterns reveal nothing about which tokenizer entries are touched. Access-count padding then removes the total access count from observable signals leaving input length as the only unavoidable leakage.

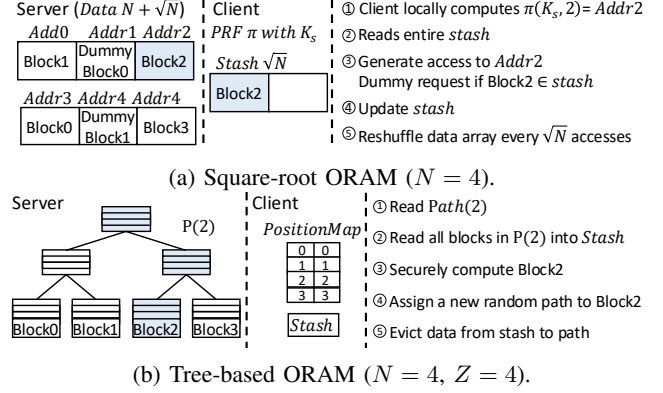


Figure 2: SqrtORAM and TreeORAM client-server model.

2. Background

2.1. Trusted Execution Environment

Trusted Execution Environment (TEE, *a.k.a.*, confidential computing) is a hardware-protected *enclave* that executes code and processes data with confidentiality and integrity via memory encryption and isolation [38], [39], [40], protecting against privileged software (e.g., a malicious hypervisor) and physical attack.

Confidential Virtual Machine (CVM). CVMs extend TEE protection to an entire VM, securing both the guest OS and its applications within a hardware-protected boundary [40], [41], [42]. Unlike process-scoped enclaves (e.g., Intel SGX [43]), CVMs avoid enclave size limits and paging overheads, supporting multi-TB memory for data-intensive workloads in most server-class CPUs [9], [40], [42], [44], [45], [46].

GPU TEE and Confidential LLM Inference. NVIDIA’s Hopper (H100) and Blackwell (B200, RTX Pro 6000) GPUs extend TEE protection to the GPU, providing device-rooted attestation [47] and per-VM isolated memory [11], [12]. Together, CPU and GPU TEEs form an end-to-end confidential stack over PCIe, and major providers including Meta, Microsoft, and Google are assembling production confidential LLM inference pipelines on top of this foundation [14], [15], [48], [49], [50], [51], [52]. TEEs are the most practical candidate for confidential LLM inference, as alternatives such as homomorphic encryption or MPC are too slow, and differential privacy cannot provide complete confidentiality against a malicious hypervisor.

2.2. LLM Tokenization

Tokenization converts human-readable text into discrete token identifiers (tokenIDs) in the LLM inference pipeline. Widely deployed decoder-only LLMs use byte-pair encoding [53] over alternatives such as WordPiece [54], and SentencePiece [55], as it handles rare and out-of-vocabulary words through iterative merging of frequent byte pairs. Our work focuses on BPE tokenizers using HuggingFace’s Tokenizers library [56] with further details in Section 3.2.

2.3. Oblivious Random Access Memory

Oblivious random access memory (ORAM) [35] is a cryptographic mechanism that conceals memory access patterns from an adversary observing the memory interface. ORAM guarantees that any two logical access sequences of the same length result in computationally indistinguishable physical access traces. Most ORAM constructions assume a client-server setting (Figure 2), in which a small trusted client maintains the ORAM metadata, while an untrusted server stores the encrypted data blocks. In the following sections, we explain two fundamental ORAM structures, square-root ORAM (SqrtORAM) and tree-based ORAM (PathORAM), using the following notations. N denotes the number of data blocks in ORAM, and stash is the stash of the ORAM. For PathORAM, L is the depth of the ORAM tree, and Z is the number of blocks per bucket, which includes both the real blocks and dummy blocks. Lastly, $\mathcal{P}(l)$ denotes the path l .

Square-root ORAM. SqrtORAM stores N data blocks and $\sqrt{N}$ additional dummy blocks in an encrypted array on the untrusted server. The client holds the pseudo-random function, (PRF) π with key K_s and a stash of size $\sqrt{N}$. To access a block $Block_a$, (1) the client performs a linear scan of stash; (2) if not found, the client performs $\text{PRF}(K_s, Block_a)$ locally and generates a *real* access request to the main array; if found, the client performs $\text{PRF}(K_s, Block_{dummy})$ locally and generates a *dummy* access (indistinguishable from the server’s view) to the main array; (3) securely computes/updates the requested data block; (4) (re)inserts it into the stash; (5) at every $\sqrt{N}$ logical accesses, an oblivious rebuild/reshuffle phase is triggered, shuffling all $N + \sqrt{N}$ blocks into a freshly permuted array using an oblivious sorting or permutation network. Each access in SqrtORAM requires a single block transfer to the server, minimizing latency and bandwidth overheads, but the rebuilding phase incurs $O(N \log^2 N)$ work.

Tree-based ORAM. PathORAM organizes N data blocks into a binary tree of height $L = \log(N)$, where each tree node is a bucket that can hold up to Z ORAM blocks on its untrusted server. Each node is always padded with dummy blocks to maintain Z ORAM blocks. The client manages a position map that records the leaf (i.e., tree path) currently assigned to each logical data block, and a small buffer called the stash. PathORAM access proceeds as follows: (1) look up the leaf $\mathcal{P}(l)$ derived from the position map; (2) read the entire path $\mathcal{P}(l)$ into the client and move the retrieved blocks into the stash; (3) securely compute/update the requested data block; (4) assign the block a fresh random leaf and update the position map; (5) evict blocks from the stash along $\mathcal{P}(l)$ within the bucket capacity limit. Because each access transfers a full root-to-leaf path, PathORAM’s bandwidth and per-access eviction logic, which require write-back into the $\mathcal{P}(l)$, can dominate when accesses are small and frequent (as in tokenizer-table lookups).

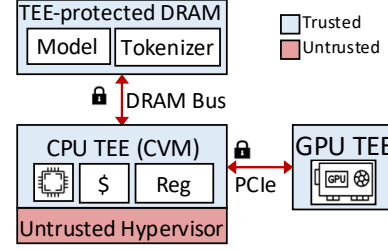


Figure 3: Threat model of a Confidential LLM Service Deployment.

3. Leakage from Tokenizer Execution Path

In this section, we focus on how the tokenizer’s execution path becomes a concrete source of information leakage even when LLM inference is protected by TEEs. We begin by formalizing the threat model, then we present a motivating example that illustrates how input-dependent behaviors in the tokenizer expose structural access patterns. Finally, we evaluate naive defenses and show that the overhead is incompatible with latency-sensitive LLM serving.

3.1. Threat Model

Figure 3 illustrates the system topology for confidential LLM serving. The CPU TEE hosts the CVM protecting the guest OS, NVIDIA driver, software stacks including PyTorch, and tokenizer; the GPU TEE securely stores model weights and activations in on-package HBM. HBM is treated as part of the trusted computing base because of its tight physical integration within the GPU package [57], eliminating any observability into access patterns within the HBM and GPU L2 cache. This is consistent with existing TEE deployments which assume the integrity of the CPU and GPU SoC packages. Additionally, all traffic between CPU TEE, DRAM, and GPU TEE over PCIe is encrypted and authenticated.

Under this architecture, we adopt the standard threat model for CPU and GPU TEEs [6], [12], [14], assuming an adversary controlling the full software stack with physical access to the cloud server. With these capabilities, the adversary can mount software-based attacks (page fault-controlled channels and prime-and-probe cache attacks) to observe DRAM access patterns at cache line granularity [16], [25], [31] and can snoop the DRAM bus directly to obtain address-level access patterns [58], [59]. The adversary can perform a calibration phase of chosen queries to the tokenizer, correlating traces across many inputs to localize the memory regions and layouts of tokenizer tables, even under per-process hash randomization. This calibration is demonstrated as the one-time token-localization stage of TDXRay [31] where it recovers the table layout offline using crafted prompts. Therefore, we conservatively assume the tokenizer table layout is known to the adversary, and layout obscurity provides no meaningful protection against access-pattern leakage.

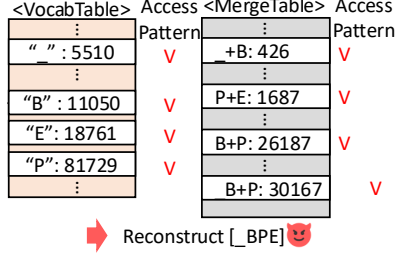


Figure 4: Access-pattern leakage during BPE Tokenization.

We distinguish two leakage channels. *Fine-grained access-pattern leakage*: a malicious hypervisor observing DRAM traces can reveal which specific vocabulary and merge-table entries were accessed, enabling exact prompt reconstruction (Section 3.2). *Coarse length-based leakage*: BPE table-lookup count, observable through ORAM access count, scales linearly with prompt byte length which is unavoidable in any practical ORAM deployment; hiding it would require padding every request to the maximum context length, incurring prohibitive overhead. OTRO targets the first channel as the more severe threat of exact reconstruction. The remaining input prompt length can be addressed independently through request length bucketing.

3.2. Case Study: Leakage in BPE Tokenizer

Compared to the largely branch-free, matrix-multiplication-dominated computation inside transformer blocks, the tokenizer is a branch-heavy, table-driven component that operates directly on the raw user prompt. Each input character or byte triggers conditional logic, hash lookups, and iterative merge decisions. This combination of *rich control flow* and *prompt-dependent memory accesses* makes the tokenizer an especially attractive target for side-channel attacks: leakage at this stage directly reveals the user’s original text, rather than intermediate embeddings or internal activations. We next explain the tokenizer process with a simple example and identify the source of leakage in it.

Figure 5 illustrates the workflow of the BPE tokenizer, consisting of the encoder step (Figure 5a) and the BPE merge algorithm (Figure 5b). The goal of the tokenizer in this example is to tokenize the input string “tiktoken is a fast BPE tokenizer.” The BPE tokenizer in Figure 5a takes the following steps. First, the pre-tokenization step takes our example input string and breaks it down into tokenizer-manageable pieces; in our example in Figure 5a, it is broken into per-word segments. The per-word segment is shown in the central tile of Figure 5a. Each word is preceded by an “_” to indicate the start of the word segment.

The next step is the encoder step (step 3). This step looks up the vocab table, which holds the learned vocabulary and its corresponding tokenIDs. For example, common words such as “fast” hit in the vocab table. But uncommon words such as “tiktoken” are not present in the vocab table. We

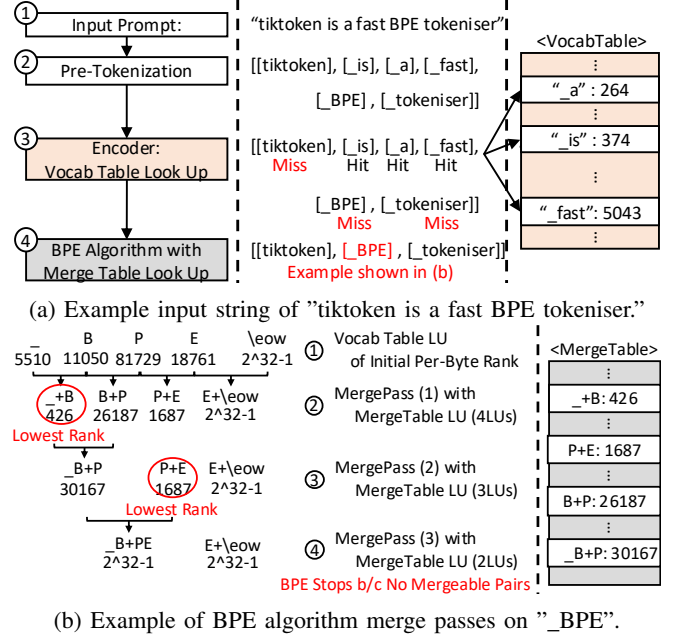


Figure 5: BPE tokenizer architecture.

illustrate which words hit and miss in the vocabulary table in the center tile of the figure under each word.

Upon failure to find a valid token in the vocab table for words, such as “tiktoken”, BPE tokenizer builds a list of tokenIDs that together approximate the target word. This step is where the BPE algorithm has to access the vocabulary table repeatedly to merge short tokens into longer tokens using multiple merge passes. We illustrate that step next.

Figure 5b shows the algorithm execution path for tokenizing the word “_BPE”. The word is first split into individual characters where each character is treated as a token. The tokenizer algorithm finds a pair of adjacent tokens (characters) with the lowest rank, as illustrated by the numerical values in the merge table (greedy path selection), at each merge pass, and merges these two tokens into a single token. Importantly, each merge modifies the current token sequence, which in turn changes the set of adjacent candidate pairs considered in subsequent passes. As a result, the merge-table lookups are inherently dynamic; after each merge, newly formed adjacent pairs must be queried against the merge table, while previously valid pairs may become invalid. Consequently, the set of merge-table queries at pass $k + 1$ cannot be determined before completing pass k , making the merge procedure inherently sequential. The word “_BPE” is decomposed into the base symbols of “_”, “B”, “P”, and “E”. Then, the algorithm consecutively chooses the adjacent pairs “_+B” and “P+E”, as they have the lowest ranks in the merge table (426 and 1687, respectively). BPE merges stop when there are no mergeable pairs, which is denoted by $2^{32} - 1$.

Consider the series of access requests shown in Figure 4 that is derived from the examples in Figure 5b, for the input word “_BPE.” Given that the model’s vocabulary and

merge tables are fixed, each observed lookup address can be mapped back to a concrete entry (a vocab token or a merge-pair with a specific rank) using an offline address-to-entry map. The vocabulary accesses reveal that the initial base symbols were materialized during the execution (e.g., “_”, “B”, “P”, and “E”). Also, the merge-table accesses reveal the candidate adjacent pairs examined at each step (e.g., “_+B”, “B+P”, and “P+E”); the selected merge can be inferred from the subsequent update-dependent accesses (e.g., “_B+P” is the next pass’s changed set of candidate pairs that were probed).

With these observations, an adversary can reconstruct the word by replaying the deterministic BPE procedure. In the first merge pass, the tokenizer probes the merge-table entries, namely “_+B”, “B+P”, and “P+E”, which let an observer recover the ranks of these candidates. Although the trace does not explicitly label which candidate was chosen, the chosen merge can be inferred from the subsequent access behavior: the tokenizer materializes the merged token, “_B” via a vocabulary lookup and updates the local neighborhood of the merged position, which changes the set of adjacent pairs that will be queried in the next pass. Observing that the next pass probes pairs consistent with “_B” (rather than “BP” or “PE”) identifies “_+B” as selected merge. Repeating this reasoning across such replays produces the same intermediate token sequence and thus the same final tokenization outcome. Because BPE tokenization is deterministic given the vocabulary and merge rules, the replay procedure described above uniquely determines the resulting token sequence. Thus, address-level vocabulary and merge table access traces are sufficient to reconstruct tokenizer outputs without observing plaintext memory contents.

4. Prompt Reconstruction on Intel TDX

The leakage of user prompt through tokenizer access pattern in Section 3.2 is realized end-to-end on Intel TDX by TDXRay [31]. TDXRay performs prompt-reconstruction attack against unmodified confidential VM performing LLM inference, recovering user prompts from the tokenizer’s memory accesses while all guest memory remains encrypted.

The attack proceeds in two steps. In the one-time setup, the adversary builds a host-side tracer from the TDX interface, which combines the page-management APIs (e.g. `TDX.MEM.RANGE.BLOCK/UNBLOCK` with a cache side-channel, pinning each access to the 64-byte cacheline it touches. Using custom prompts, it then localizes the tokenizer’s table entries in the guest’s physical memory, exploiting the fact that the tokenizer’s table layout is fixed after initialization. Because this layout is static until the victim process restarts, the cost is amortized over the lifetime of the inference.

Once the table is localized, each victim prompt leaks its entire plaintext. The adversary records the cache-line granularity access trace of the tokenizer tables and replays the deterministic lookup procedure. This is the same reconstruction argued in Section 3.2 to recover the prompt.

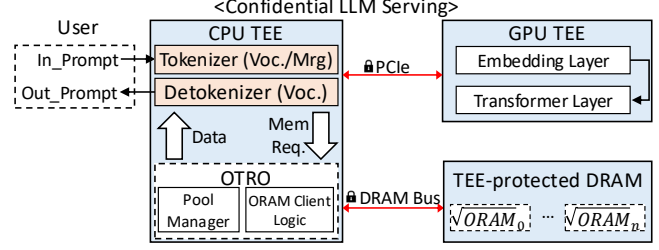


Figure 6: Overview of OTRO system.

TDXRay reports over 90% reconstruction similarity across model families. This result proves that the tokenizer is a demonstrated, highly-vulnerable prompt-reconstruction channel under a realistic host adversary that even survives the TEE protections. Thus, defending against it requires making the tokenizer’s table-access pattern independent of the input, which we develop in Section 5.

5. OTRO: Oblivious Tokenization Path with Square-Root ORAM

In this section, we introduce OTRO, an *efficient and oblivious tokenization pathway* built on top of SqrtORAM and tailored to BPE tokenizers in LLM inference serving.

5.1. OTRO Overview

Directly embedding SqrtORAM into the LLM tokenization pipeline presents two challenges: (i) each instance serves only $\sqrt{N}$ accesses before requiring a rebuild, and (ii) the rebuild costs $O(N \log^2 N)$, which is prohibitive on the critical path. OTRO overcomes these obstacles by exploiting the read-only nature of tokenizer tables and by decoupling accesses from costly rebuilds via three ideas.

Pooled SqrtORAM Replication: OTRO replicates the static vocabulary and merge tables across a *pool* of read-only SqrtORAM instances, effectively increasing the $\sqrt{N}$ -access limit. *Epoch-Based Rotation and Background Rebuild:* an epoch-based rotation policy routes accesses to instances with remaining $\sqrt{N}$ -access budget, while depleted instances rebuild asynchronously in the background, so that rebuilds never block foreground tokenization. Dummy accesses pad each epoch to its $\sqrt{N}$ boundary, reducing the observable transcript to a coarse epoch count. *Chunked Tokenization with KV-Shared Prefill:* long prompts are split into chunks whose tokenization can be interleaved with GPU prefill using a shared KV cache, reducing the active SqrtORAM instance count and providing slack to hide rebuild latency.

Figure 6 illustrates the overall OTRO system deployed under a CVM. The tokenizer executes in the CPU TEE, while the embedding and Transformer layers (including both prefill and decoding) execute in the GPU TEE. Each client session receives an independent SqrtORAM pool keyed by identifier *cid*, ensuring full isolation among clients; no client’s accesses influence another’s epoch counters or

Algorithm 1 Per-client Epoch-based Scheduler

```

1: Input:  $cid$ : client identifier,  $request$ 
2: Global:  $Pool_{cid}[1..n_{pool}]$ ,  $ctr_{cid}[1..n_{pool}]$ ,  $state_{cid}[1..n_{pool}]$ ,
 $active_{cid}$ ,  $EPOCH\_LIMIT \leftarrow \sqrt{N}$ 
3: function GLOBALACCESS( $cid, request$ )
4:    $active \leftarrow active_{cid}$  if defined, else smallest  $j$  s.t.  $ctr_{cid}[j] = 0$ 
5:    $active_{cid} \leftarrow active$ 
6:    $inst \leftarrow Pool_{cid}[active]$ 
7:    $response\_msg \leftarrow READ(inst, request)$ 
8:    $ctr_{cid}[active] \leftarrow ctr_{cid}[active] + 1$ 
9:   if  $ctr_{cid}[active] == EPOCH\_LIMIT$  then
10:     $state_{cid}[active] \leftarrow REBUILDING$ 
11:    LAUNCHREBUILDASYNC( $cid, active$ )
    {async: oblivious shuffle, reset ctr, set READY}
12:     $active_{cid} \leftarrow (active \bmod n_{pool}) + 1$ 
13:   end if
14: return  $response\_msg$ 

```

rebuild schedule. The pool manager selects instances via the epoch-based rotation policy and triggers asynchronous rebuilds for exhausted instances, keeping rebuild work off the critical path. Since per-client pools are independent, multi-client throughput scales with GPU scheduling, so the single-client evaluation in Section 7 captures the full OTRO-specific performance.

5.2. SqrtORAM Pool Access

The key observation in the LLM tokenizer during inference is that tokenization and detokenization issues only read operations to the vocab/merge tables. The read-only access pattern indicates the rebuild never needs to reconcile updated blocks allowing OTRO to *decouple* the strict rebuild-after- $\sqrt{N}$ -access requirement of SqrtORAM. OTRO maintains a pool of independently permuted SqrtORAM instances that hold the same vocab/merge data, so while one instance rebuilds asynchronously, other continues serving requests uninterrupted.

Initialization. The LLM tokenizer’s vocab and merge tables are publicly available, but we seek to obfuscate subsequent access patterns by constructing multiple independently permuted SqrtORAM instances. When a new client session begins, OTRO assigns the client a unique cid and uploads the plaintext tokenizer data into TEE-protected DRAM. OTRO concatenates the vocab and merge table into a single flat array, then generates P independent permutations using per-instance keys ($k_i, 0 \leq i < P$), instantiating P SqrtORAM structures each with a distinct random layout. Because the tokenizer data is public, the obliviousness of future accesses relies solely on access-pattern indistinguishability provided by each SqrtORAM instance rather than on the layout obscurity. OTRO retains all ORAM metadata, including PRF, random key (k_i), stash, and access counters for each ORAM instance in its TEE-protected memory. Furthermore, with per-client pools, access patterns from distinct clients are fully isolated.

Access Request Serving. During LLM inference, the tokenizer’s memory request is submitted to the OTRO pool manager, which selects a SqrtORAM instance from the

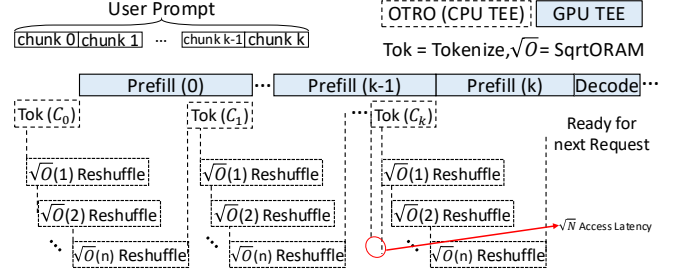


Figure 7: Chunked tokenization with asynchronous reshuffle.

client’s dedicated pool. We deploy an epoch-based instance rotation policy, Algorithm 1, where each SqrtORAM instance is fully utilized, serving $\sqrt{N}$ accesses before being scheduled for rebuild. The manager maintains a pool of SqrtORAM instances and a pointer to the currently active instance. Each instance serves up to $\sqrt{N}$ accesses in an epoch, during which all incoming requests are mapped to that instance, independent of the address being accessed. Once an instance reaches its per-epoch access limit, the manager launches an asynchronous rebuild (LAUNCHREBUILDASYNC) for that instance in a detached thread and advances the active pointer to the next instance in the pool. The choice of the instance is a deterministic function of the global access count; an adversary can see when OTRO switches instances, but the switching of instances is a function of “global access count”, which is already leaked in any practical ORAM designs. Thus, the decision is independent of the logical sequence and does not encode any information about the data being accessed.

Access Count Padding. The exact ORAM access count within an epoch is observable to the hypervisor and leaks fine-grained length information beyond what byte length already reveals. Since each epoch naturally spans exactly $\sqrt{N}$ accesses, OTRO pads dummy accesses to the next $\sqrt{N}$ boundary at epoch completion, causing at most $\sqrt{N} - 1$ additional dummy accesses. This replaces the exact access count with the coarser epoch number, dramatically reducing the residual access-count leakage, which is analyzed in Sec 7.5.

5.3. Chunked Tokenization

As shown in Figure 7, we introduce chunked tokenization to effectively hide rebuild latency in variable-length prompt input scenarios while minimizing the number of SqrtORAM instances. Supporting tokenization for variable-length input prompts would ideally require a pool sized to the model’s maximum context length, which can expand to the order of hundreds of instances for long-context models like Llama-3.1 (128K tokens). Chunked tokenization effectively reduces the number of replicas by interleaving tokenization and GPU prefill: the pool need only be large enough to cover one chunk at a time rather than the full prompt.

The number of instances can be determined either by the model’s maximum context length or by the ratio between rebuild and $\sqrt{N}$ -access latency. Each LLM has a maximum context length of $L_{\max_ctx}$ tokens. We can set the pool size, $n_{\text{pool}} = \min(\lceil \frac{L_{\max_ctx} * \lambda_{\text{att}}}{A_{\text{oram}}^{\max}} \rceil, \lceil \frac{T_{\text{reshuffle}}}{T_{\text{access}} * A_{\text{oram}}^{\max}} \rceil)$ where λ_{att} is the ORAM access rate per token, $T_{\text{reshuffle}}$ is the per-instance rebuild time, and T_{access} is the per-lookup latency excluding rebuild.

OTRO partitions the input prompt into multiple smaller chunks that are tokenized and fed into the LLM pipeline. The chunking is strictly an internal scheduling mechanism occurring at pre-tokenization boundaries (*i.e.*, whitespace-delimited words) and does not change the logical behavior of the tokenizer or the model; BPE merges never cross the chunk boundaries, producing a token sequence bit-for-bit identical to tokenizing the full input at once. Additionally, for correctness, special tokens, such as beginning-of-sequence and end-of-sequence, are not inserted at the chunk boundaries. With incremental prefill over a shared KV cache, the model sees a single contiguous token sequence of tokenIDs identical to the non-ORAM baseline; OTRO’s chunking affects only performance and memory behavior, not semantics.

Chunk size $\text{chunk}_{\text{tok}}$ is chosen with explicit awareness of GPU utilization and ORAM rebuild time so that the prefill duration of the chunk overlaps with the rebuild phase of the next chunk: $T_{\text{gpu,prefill}}(\text{chunk}_{\text{tok}}) > T_{\text{reshuffle}}$. This ensures that once the tokenization of the chunk ends and the rebuild phase is launched, the entire rebuild phase completes while the GPU is executing the prefill stage. We select the chunk size so that each chunk’s GPU prefill interval serves as a “coverage window” that amortizes the SqrtORAM rebuild cost, allowing the system to maintain high GPU utilization while keeping SqrtORAM-induced stalls off the critical path of the LLM pipeline.

5.4. Detokenization

The de-tokenization process converts the tokenIDs generated by LLM decoding into human-readable text using a reversed vocab table. Obfuscating access patterns to this structure, therefore, plays an important part in guarding output prompts from leakage. We apply our ORAM-based obfuscation, OTRO to detokenizer accesses. The main difference lies in ORAM block organization; each ORAM block needs to be padded to the maximum vocabulary length in the reversed vocab table. Without padding, the word length could leak from the returned message length of ORAM access. Thus, compared to OTRO in the tokenization path, we will have a fewer blocks (N), but a bigger data size per block.

Similarly to overlapping tokenization and LLM prefill, we overlap detokenization and the LLM decode stage. In contrast to the prefill stage, which requires the entire tokenized result of the input prompt, the decode stage emits output tokens every few tens of milliseconds. Thus, the number of instances required by OTRO can be dramatically reduced,

as the $\sqrt{N}$ accesses occur over hundreds of milliseconds to seconds, in which the other instance can finish its rebuild.

6. Security Analysis

Threat Model. We consider a probabilistic polynomial-time adversary $\mathcal{A}$ that controls the untrusted software stack and can observe all physical DRAM accesses generated by the CVM. Due to TEE memory protection and encryption, the adversary cannot observe plaintext DRAM contents, cryptographic keys, or internal CVM state, and it cannot tamper with execution inside the CVM.

Security Goal. The security goal of OTRO is to protect the table-dependent memory behavior of tokenization and detokenization. Concretely, even if an adversary can observe the CVM’s physical DRAM trace, they should learn nothing about which logical entries in the vocabulary table or merge table are accessed. In other words, OTRO provides an access-pattern confidentiality guarantee for tokenizer-table lookups, which is the primary channel that links DRAM addresses to specific tokens or merges. This guarantee is intentionally scoped to the ORAM-protected table interface, rather than the entire tokenizer implementation. Consistent with prior ORAM-based systems [36], [37], [60], we do not attempt to eliminate side channels that stem from computation-level effects (*e.g.*, timing variability or microarchitectural vulnerability) that may arise from specific code patterns or implementation details; analyzing and hardening such channels requires a different methodology and is orthogonal to OTRO’s design. Likewise, OTRO does not protect against input-structural leakage that is either unavoidable or application-dependent, including the processing time, the length of the input sequence, and the total number of tokenizer table lookups by default; with padding enabled, only the coarser epoch count is revealed.

Observable Transcript. The observable transcript consists of the sequence of physical DRAM accesses arising from i) tokenizer and detokenizer table lookups and ii) background rebuild operations. Public configuration parameters, including vocabulary size, table layout, pool size, chunk size, and the maximum context length of the model, are explicitly revealed. Without access count padding, the observable transcript includes the total number of accesses and the number of processing epochs. With access-count padding enabled, only the epoch count $\lceil \text{access_count} / \sqrt{N} \rceil$ is revealed, since dummy accesses fill each epoch to the $\sqrt{N}$ boundary before rotation. For example, OTRO may reveal that a given input triggered 3014 table accesses processed across two chunks, but it should not reveal which logical entries of the tokenizer tables were accessed or the frequency of such accesses.

Epoch-based Scheduling and Instance Rotation.

Let $j(i) = \lfloor \frac{i-1}{\sqrt{N}} \rfloor \bmod n_{\text{pool}} + 1$ denote the SqrtORAM instance selected for the i -th logical access. By Algorithm 1, $j(i)$ is a deterministic function of i and public parameters $\sqrt{N}$ and n_{pool} ; it does not depend on the logical address being accessed. Since rebuild operations are triggered solely when the per-instance access counter reaches $\sqrt{N}$, their

TABLE 1: Tokenizer table configuration.

Tokenizer Config	LlaMA-3.1	Qwen3	Gemma-3	Phi-3
Vocabulary Size	128,000	151,643	262,144	32,000
Merge Rules	280,147	151,387	514,906	61,249
Max String Length	256	256	93	48
Context Length	128,000	32,000	128,000	128,000

timing is likewise determined by the aggregate access count. Thus, instance rotation and rebuild scheduling introduce no additional input-dependent leakage beyond what is already captured in the observable transcript.

Security Proof Sketch.

Theorem 1 (Execution Trace Indistinguishability). *For any two executions inducing the same number of tokenizer accesses, no probabilistic polynomial-time (PPT) adversary can distinguish their physical DRAM traces with non-negligible advantage.*

Proof Sketch. By Lemma 1 (Appendix B), the per-access DRAM trace distribution is identical across both executions: the instance is selected deterministically from i and public parameters and the PRF maps the logical address to a location uniform over $N + \sqrt{N}$ positions. By Lemma 2, traces from distinct accesses are mutually independent since PRF evaluations are keyed independently per instance, and reshuffles follow a fixed public schedule. By independence, the joint distribution factors into a product of identical per-access distributions, so the distinguishing advantage is negligible. Together, Lemmas 1 and 2 establish that the complete DRAM trace distribution is identical across any two executions with the same access count, giving Theorem 1. Full proofs are in Appendix B.

7. Evaluation

7.1. Experimental Setup and Implementation

System Setup. We evaluate on an Intel Xeon Gold 6548Y+ and an NVIDIA H100 GPU (Driver 550.163.01). Host runs Ubuntu 25.04, and the guest CVM (Intel TDX) runs Ubuntu 24.04 with 64 vCPUs and 256 GB memory. Because we implement ORAM logic inside unmodified TDX, we inherit the deterministic AES-XTS behavior of TME; integrating Obelix-like probabilistic memory encryption would remove ciphertext determinism and is considered orthogonal to our work.

Implementation. We prototype OTRO in HuggingFace Tokenizers (v0.22), redirecting each memory request to OTRO. Chunked tokenization works as a scheduling module that interleaves tokenization and prefill above nano-LLM’s [61] existing KV-cache mechanisms. We disable the tokenizer’s software merge cache in all configurations as its data-dependent hit/miss pattern leaks additional prompt information outside of the ORAM-protected path. We compare OTRO with Baseline (no obfuscation), PathORAM [36], and naive SqrtORAM [35] ($np=1$), and evaluate two OTRO

variants: 1) GPU overlap (GO), which overlaps rebuilds with GPU computation but stalls if all instances are exhausted, and 2) OTRO (GO+CT) which additionally applies chunked tokenization to eliminate the stalls. To probe the no-stall regime, we also provision a large upper-bound pool ($np=300$); we derive the minimum pool size using $n_{pool} \geq \lceil T_{rebuild}/T_{epoch} \rceil$, which yields approximately 201, 59, 146, and 419 for Llama-3.1, Qwen3, Gemma-3, and Phi-3, respectively. For Phi-3, whose epoch consumption time is particularly short, a large pool (≈ 419) would be required to strictly avoid stalls; however, due to its shorter rebuild latency, observed stall durations remain small in practice.

Metrics, Models and Benchmarks. We measure time-to-first-token (TTFT), tokenizer memory, and initialization cost for Llama-3.1-8B [27], Qwen3-8B [29], Gemma3-4B [26], and Phi-3-mini [28] under PyTorch-2.6. These models cover a range of vocabulary sizes and merge-table configurations (Table 1); since ORAM overhead depends primarily on tokenizer vocabulary size rather than model depth, they are representative without requiring larger variants. Input prompt length is swept from 1500 to 6500 words, where each word yielding 3–5 tokens. Lastly, we note that OTRO’s per-client design (Sec 5.1) ensures that concurrent multi-client behavior is architecturally independent across clients; the only shared resource is the GPU, which affects baseline and OTRO equally. Therefore, the single-client prompt-length sweep presented here captures the full OTRO-specific performance behavior, and multi-client throughput scales with GPU scheduling rather than ORAM design choices.

Deployment Considerations. OTRO requires no changes to model weights, CUDA kernels, or transformer architecture. Chunked tokenization adds a thin scheduling layer above the serving framework’s existing prefill and KV-cache mechanisms, with no retraining or model reconfiguration required, and thus, OTRO can be modularly deployed, agnostic to the underlying serving framework.

7.2. Offline Tuning for CPU-GPU Overlap

Before running our main experiments, we perform an offline profiling phase to determine how to best overlap the rebuild phase of OTRO’s SqrtORAM instances with the model’s prefill phase. For each model, we sweep the chunk size for tokenization and the number of SqrtORAM instances in the pool and profile them together with the GPU prefill kernels. The goal is to select a configuration that maximizes overlap between CPU-side work (chunked tokenization and SqrtORAM rebuilds) and GPU prefill.

Maintaining GPU utilization. During the offline tuning, we monitor GPU utilization in the prefill phase to avoid configurations where aggressive chunking results in GPU SM starvation. Very fine-grained chunk sizes can reduce the amount of work (tokens) fed per prefill kernel launch and lower GPU occupancy, offsetting the benefits of overlap. We discard the configurations that noticeably degrade prefill utilization and choose the best-performing setting that both preserves high GPU utilization and provides strong overlap

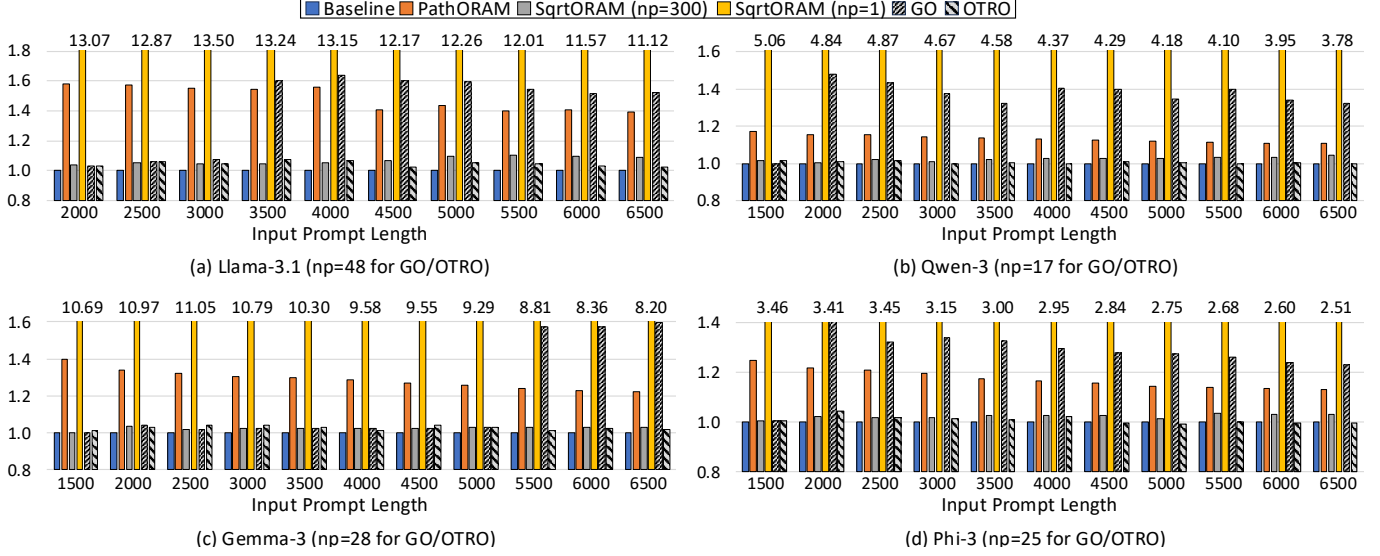


Figure 8: Normalized TTFT in Llama3.1, Qwen3, Gemma-3, and Phi-3.

CPU-GPU overlap. Our profiling shows that for Llama-3.1, using a prompt chunk of approximately 3000 words (≈ 9000 tokens after tokenization) provides enough CPU slack to complete the rebuild from the current epoch and tokenize the next chunk, and overlap with the GPU prefill. Offline profiling selects chunk sizes of ~ 3000 , ~ 1700 , ~ 5000 , and ~ 1500 words for Llama-3.1, Qwen3, Gemma-3, and Phi-3, respectively, maintaining GPU utilization within 83–100%.

Detokenizer ORAM Config. During the decode stage, our profiling shows an average GPU utilization of 51–65% and memory utilization of 41–49% across all models, with approximately 20–30 ms of compute per generated token. Given this headroom, we find that a pool of just two SqrtORAM instances is sufficient: their rebuild work can be entirely overlapped with the decode kernels, so the cost is effectively hidden in the decoder phase.

7.3. Time-To-First-Token Latency

Figure 8 presents the normalized TTFT over the baseline of no ORAM obfuscation in Llama-3.1, Qwen-3, Gemma-3 and Phi-3. Figure 9a shows the percentage of tokenization latency in TTFT.

Impact of ORAM-based Tokenization on Llama-3.1.

We first evaluate the end-to-end effect of ORAM-based tokenization on TTFT for Llama-3.1. Across input prompt length, the normalized TTFT reflects the latency overhead of the tokenizer-only implementation depicted in Figure 1b. PathORAM implementation consistently degrades TTFT by an average of $1.48\times$, as tokenizer latency occupies nearly 35.73% within the entire TTFT. The result reflects the high per-access cost of $5.16\times$ in the tokenizer-only experiment. For naive SqrtORAM (np=1) implementation, TTFT increases by an average of $11.12\times$, in which the tokenization time dominates TTFT with 92.42%; the rebuild cost

TABLE 2: Additional memory requirements (in MB) to support different ORAM schemes in tokenizer and detokenizer stages.

Stage	Model	Memory Overhead (MB)		
		PathORAM	SqrtORAM	OTRO
Tokenizer	LLaMA-3.1	57.9	4.84	285.9
	Qwen3	37.5	4.5	46
	Gemma3	203.41	12.52	332.0
	Phi3	14.5	1	30.5
Detokenizer	LLaMA-3.1	571.4	3.73	7.68
	Qwen3	260.3	2.5	8.34
	Gemma3	274.8	11.0	22.09
	Phi3	17.9	3.0	4.0

for naive SqrtORAM implementation occupies 99% of the tokenization time.

SqrtORAM (np=300), which provisions the number of SqrtORAM instances to an upper-bound, depicts an ideal scenario where SqrtORAM does not pay any rebuild penalties during the tokenization. Compared to baseline, the results show a 6.90% increase in TTFT; the average access latency increases by $1.26\times$ in Figure 1b. The tokenization time is only 10.86% of the total TTFT. GO (np=48) performs near-optimal from 2000 to 3000 input prompt length, but spikes after 3500. The main reason for this spike is that once the access limit is reached for all instances in the pool, the next tokenization must wait until at least one instance is free. While the rebuilds are overlapped and the system does not pay the full price as in naive SqrtORAM (np=1), GO (np=48) pays an average of 57% increase in TTFT. OTRO, which is depicted as GO+CT (np=48) in the figures, performs similarly to the ideal case of SqrtORAM (np=300) in smaller input settings. However, once input length exceeds 4000, OTRO reduces TTFT by 5% compared to the ideal. As chunk tokenization encodes the first 3000

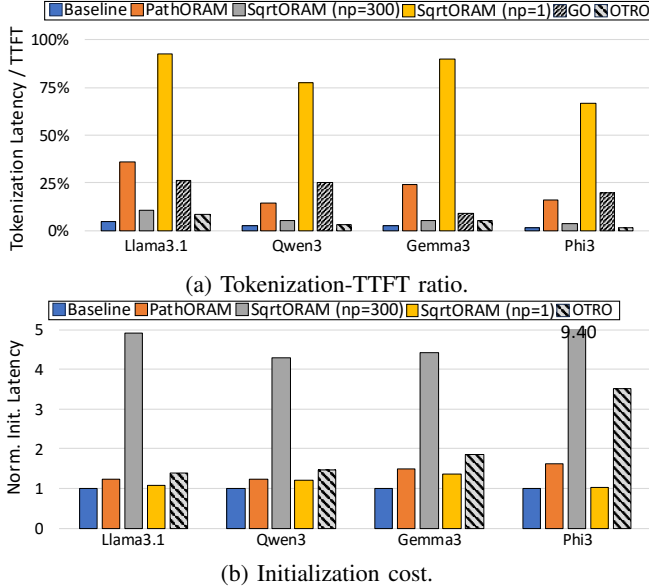


Figure 9: (a) Tokenization overhead as a fraction of TTFT and (b) Normalized tokenizer initialization cost.

words and overlaps the rest with GPU computation, we do not pay the extra tokenization cost induced by SqrtORAM. Across our evaluation, OTRO increases TTFT by only 4.59% on average compared to the baseline. This small overhead leaves TTFT dominated by the model execution (91.41%) rather than tokenization (5.58%), so the cost of oblivious tokenization is modest in practice.

Cross-Model Trends. Across Qwen3, Gemma-3 and Phi-3, the qualitative trends mirror Llama-3.1: naive SqrtORAM ($np=1$) incurs the worst overhead, GO spikes at long prompts once all instances exhaust their $\sqrt{N}$ budget, and OTRO (GO+CT) stays near the ideal SqrtORAM curve. The key difference is that tokenization accounts for under 3% of baseline TTFT for all three models, yet the naive PathORAM and SqrtORAM ($np=1$) inflates this significantly. Qwen3 generates more tokens per word (4.89 on average) due to its smaller merge table, lengthening GPU prefill and further diluting tokenizer overhead; OTRO limits TTFT increase to just 0.5% (tokenizer accounts for 3.2% of TTFT). Gemma-3 has 2x larger vocab and merge tables than Llama-3.1 but allocates more budgets to non-English scripts and its smaller model size (4B) keeps GPU prefill time comparable to Llama-3.1; GO’s long-prompt TTFT spike is avoided by OTRO’s chunked tokenization overlap. Phi-3 has the smallest absolute TTFT due to its compact tokenizer and model, but its normalized TTFT trends and ORAM configuration ordering are consistent with other models.

Access-Count Padding Overhead. Access-count padding adds at most $\sqrt{N}-1$ dummy accesses per epoch. Given our measured per-access latency of $\sim 3.2\mu s$ and $\sqrt{N} \leq 881$ across all evaluated models, the worst-case padding overhead is $881 * 3.2\mu s \approx 2.8ms$ per epoch; this is less than 0.4% of the Phi-3 rebuild latency (116 ms) and negligible

relative to TTFT. Thus, we omit padding overhead from TTFT figures.

Detokenization ORAM Latency. The ORAM that protects the detokenization vocab table stores longer data blocks, since each token’s textual form is padded to the maximum vocabulary string length (e.g., Llama-3.1’s longest vocab entry is 256 B). This detokenization ORAM touches its table only once during TTFT and once per decoding step to map generated token IDs back to text, so its cost is amortized over the entire decoding phase. We do not observe any systematic impact on TTFT or decoding throughput attributable to detokenization ORAM. The baseline detokenization throughputs are 21.23, 29.61, 33.08, 23.85 ms/token for Llama-3.1, Qwen3, Gemma-3, and Phi-3, respectively. Detokenizer with PathORAM and SqrtORAM shows 21.53 and 21.34 ms/token for Llama-3.1, 29.61 and 30.14 ms/token for Qwen3, 34.28 and 33.56 ms/token for Gemma-3, and 23.52 and 22.97 ms/token for Phi-3. Lastly, OTRO for detokenizer performs 23.10, 29.56, 33.55, and 21.38 ms/token for Llama-3.1, Qwen3, Gemma-3, and Phi-3, respectively. All of these differences are within the normal variance of the end-to-end decoding latency, which is on the order of 20–40 ms per step. Thus, we conclude that detokenization ORAM latency is negligible in the overall LLM pipeline.

7.4. System Level Cost

Memory Footprint. We measure end-to-end memory impact using process resident set size (RSS), recorded after (1) loading the model and baseline tokenizer, and (2) constructing the ORAM structures; then, we report the difference, as shown in Table 2. The baseline, non-oblivious tokenizer uses 60.1 MB, 92.0 MB, 137.0 MB, and 16.0 MB of memory for Llama-3.1, Qwen3, Gemma-3, and Phi-3, respectively, to store both the tokenizer and detokenizer tables inside the TEE. PathORAM increases the tokenizer footprint due to its tree layouts and dummy blocks, whereas SqrtORAM with a single instance remains within ≈ 12 MB of the baseline. Our proposed design, OTRO, requires an extra 285.9 MB for Llama-3.1 (48 instances), 46.0 MB for Qwen3 (17 instances), 333.0 MB for Gemma-3 (28 instances), and 30.5 MB for Phi-3 (26 instances). Since OTRO instantiates an independent pool per client, the memory overhead scales linearly with the number of concurrent clients; at under 0.5 GB per client, a 256 GB CVM can support tens of concurrent clients without memory pressure. For detokenizer, OTRO keeps only two SqrtORAM instances, resulting in 7.68 MB, 8.34 MB, 22.9 MB, and 4.0 MB for Llama-3.1, Qwen3, Gemma-3, and Phi-3, respectively, and we observe no systematic impact on decoding throughput across all models.

Initialization Latency. Figure 9b depicts the one-time initialization cost of installing ORAM metadata and the tokenizer tables at model load time. For Llama-3.1, the baseline initialization is ≈ 0.46 s, and OTRO increases this to ≈ 0.64 s (39.7% increase). The initialization cost grows

TABLE 3: Residual leakage (20K ShareGPT, Llama-3.1). ℓ = byte length. Higher H = more privacy.

Configuration	$H(\text{prompt} \text{obs})$	$I(\text{access_count}; \text{prompt} \ell)$
No ORAM	0.00 bits	—
OTRO (w/o pad)	1.60 bits	3.94 bits
OTRO	5.49 bits	0.04 bits
ℓ only	5.53 bits	0.00 bits

by 46.9% (from ≈ 0.40 s to ≈ 0.59 s), 86.3% (≈ 1.25 s to ≈ 2.34 s), and 351.9% (≈ 0.04 s to ≈ 0.15 s) for Qwen3, Gemma-3, and Phi-3. Since initialization happens once per model load, we do not treat the initialization cost as a major system bottleneck.

7.5. Residual Leakage Quantification

Modeling. We quantify the information available to an adversary observing OTRO’s metadata under a closed-world model: we sample 20K prompts from the ShareGPT corpus, tokenize with Llama-3.1’s BPE tokenizer, and model prompt selection as uniform over the corpus ($H(\text{prompt}) = \log_2(20,000) = 14.29$ bits) of prior uncertainty. This closed-world assumption overstates what the adversary can infer, as real prompt spaces are vastly large; our analysis is a **conservative upper bound** on adversary advantage, not a lower bound on security.

Observables. After ORAM obfuscation, the adversary’s non-trivial observables reduce to two scalar quantities: prompt byte length and total ORAM access count, both inferable from the DRAM trace. Since BPE table lookups scale linearly with input length ($R^2 = 0.9915$, $\text{access_count} = 3.20 \times \text{byte_length} + 11.26$), the adversary can directly invert the access count to recover prompt length. Epoch count is a deterministic function of total access count and the public parameter $\sqrt{N}$, and chunk count is a deterministic function of total access count and public chunk size; neither contributes independent information beyond access count. Furthermore, OTRO fuses the vocabulary and merge tables into a single ORAM structure, making individual vocab and merge table accesses physically indistinguishable. The adversary therefore cannot observe per-word merge pass counts or distinguish vocab lookups from merge lookups; only the aggregate access count is visible.

Entropy analysis. We model prompt selection as uniform over the corpus, giving the adversary an initial uncertainty of $H(\text{prompt}) = 14.29$ bits, equivalent to 20,000 equally-likely candidates. Observing byte length alone resolves 8.75 bits, leaving $H(\text{prompt} - \text{byte_length}) = 5.53$ bits of remaining uncertainty. Observing the access count in addition resolves a further 3.94 bits ($I(\text{access_count}; \text{prompt}|\text{byte_length}) = 3.94$ bits), leaving $H(\text{prompt} - \text{byte_length}, \text{access_count}) = 1.60$ bits, approximately 3 candidates on average. Critically, 8.75 bits of the 12.69 bits the adversary resolves come from byte length alone, a channel that is independent of OTRO and unavoidable in any practical ORAM deployment since the total access count is a deterministic function of input length.

OTRO’s access count contributes only 3.94 additional bits beyond what byte length already reveals, and the near-perfect linear relationship between byte length and access count ($R^2 = 0.9915$) confirms that these two observables are largely redundant.

Length-stratified results. Table 3 reports the adversary’s residual uncertainty stratified by prompt length. Short prompts are most private: for prompts under 100 bytes, 2.49 bits of uncertainty remain because many distinct short prompts share similar byte lengths and access counts, limiting the adversary’s ability to distinguish among them. As prompt length increases, byte length becomes increasingly identifying on its own. For prompts exceeding 500 bytes, $H(\text{prompt} - \text{byte_length})$ drops to 1.10 bits or below, meaning the adversary retains almost no uncertainty even before access count is considered. The near-zero residual uncertainty for long prompts is therefore driven primarily by byte length observability, not by OTRO’s access count.

Access-count padding. The 3.94 bits contributed by the raw access count beyond byte length can be further reduced through a natural extension of OTRO’s epoch structure. Since each SqrtORAM epoch already spans exactly $\sqrt{N}$ accesses before instance rotation, OTRO can optionally pad dummy accesses (Section 5.2), revealing only the epoch number $\lceil \text{access_count} / \sqrt{N} \rceil$ rather than the exact access count. This coarsening is consistent with OTRO’s existing design: the adversary already observes instance switches at epoch boundaries, so the epoch number is an already-observable quantity that carries no additional information.

Table 3 shows the effect of padding. The additional leakage beyond byte length drops from 3.94 bits to just 0.04 bits — within 0.04 bits of the theoretical minimum achievable by any system that leaks byte length (0.00 bits). Residual uncertainty $H(\text{prompt}|\text{observable})$ rises from 1.60 bits to 5.49 bits, recovering 38.4% of the original prompt entropy compared to 11.2% without padding. The per-bucket improvement is particularly striking for short prompts, where additional leakage from access count drops from 4.48 bits to 0.00 bits, and for medium prompts (100–500 bytes), where it drops from 4.07 bits to 0.11 bits. For long prompts (>500 bytes), byte length already dominates and padding contributes negligible additional benefit, consistent with $H(\text{prompt} | \text{byte_length})$ being near zero in that regime regardless.

Comparison to unprotected baseline. Without ORAM protection, the adversary observes the complete sequence of vocabulary and merge-table indices, enabling full prompt reconstruction as demonstrated in Section 3.2, leaving $H(\text{prompt} - \text{full trace}) \approx 0$ bits of residual uncertainty; the adversary identifies the prompt exactly regardless of its length. Table 3 summarizes the leakage under each configuration. OTRO without padding raises this residual uncertainty from 0 to 1.60 bits overall; OTRO with access-count padding raises it further to 5.49 bits. In both configurations, OTRO replaces the fine-grained, per-token access pattern (which enables exact reconstruction) with coarse, length-correlated metadata. The residual identification risk

for long prompts exists in any system that leaks input length, which every practical ORAM does, and is not a weakness introduced by OTRO.

8. Related Work

Side-channel Attacks in CVM. Side-channel attacks against TEEs in cloud environments are more powerful than non-TEE-based attacks. The attackers can access privileged interfaces (e.g., performance counters [62] and power consumption analyzers [63]) and intentionally control global resources [64], [65], [66], [67], [68] to gather more fine-grained information on the TEE. In particular, ciphertext side-channel attacks against AMD SEV allow the attackers to monitor ciphertexts to infer secrets [16], [25]. Current research demonstrates that side-channel attacks are serious threats to TEE environments. Thus, countermeasures to protect data and code running inside TEEs are necessary.

ML Embedding Table. [32], [33] apply ORAM to mitigate access-pattern leakage in embedding tables in ML workloads and [34] uses deep hash embedding as an alternative. These techniques operate after tokenization, on tokenIDs, whereas OTRO targets the tokenizer.

Private Information Retrieval and Oblivious Map. PIR [69], [70] provides protocol-level index hiding that allows a client to retrieve a database item without revealing the queried index to the server. PIR protects the logical query at the protocol layer, but does not inherently guarantee memory-oblivious execution under a low-level side-channel adversary observing page faults or DRAM traces. Oblivious maps [71], [72] provide a key-value abstraction built on top of ORAM that hides which key is accessed during lookup or update. While oblivious maps present a higher-level interface, their security ultimately relies on the underlying ORAM’s access-pattern indistinguishability. In contrast to PIR’s protocol-level query privacy and oblivious maps’ key-value abstraction, our work operates at the memory-access abstraction within a CVM.

Traditional ORAM. Many ORAM works like [60], [73], [74], [75], [76], [77], [78], [79], [80], [81], [82] target the traditional client-server model without TEEs. Our system assumes ORAM inside TEEs; thus, these works are orthogonal to ours.

TEE-based ORAM. ZeroTrace [83], Obliviate [84], Oblix [71] and [32], [85], [86] are based on an earlier generation of TEEs, Intel SGX [87]. With memory constraints of SGX, neither SqrtORAM or PathORAM cannot be placed inside the TEE-protected memory. Thus, operate in a different regime from VM-based TEEs. PRO-ORAM [88] builds on SqrtORAM and overlaps accesses with rebuilds to support read-only workloads but targets SGX-enabled cloud storage servers and optimizes constant-latency access to large files in a network setting. In contrast, OTRO uses a pool of SqrtORAM instances as a fine-grained protection layer for tokenizer-table lookups inside a CVM, with the objective to minimize TTFT for real-time inference. As a result, while both systems exploit read-only data and

pipelined rebuild, they address different deployment models, threat surfaces, and latency objectives.

A recent work, Menhir [89], is, to our knowledge, the first system to leverage VM-based TEE in implementing ORAM logic, with Intel TME protecting the ORAM structure in TEE-protected DRAM. Obelix [90] identifies the deterministic behavior in AES-XTS-based Intel TME as a limitation and resolves it by attaching a 64-bit counter that adds probabilistic variation to AES-XTS, effectively randomizing the ciphertext for repeated accesses to the same physical location. These works are complementary to our design, hardening the security guarantees in ORAM logic.

9. Conclusion

Tokenizers inside TEEs remain a real, exploitable source of side-channel leakage. Greedy BPE tokenization yields deterministic, data-dependent table accesses, producing DRAM address traces that a privileged adversary can observe. Recent work has turned this into an end-to-end attack that reconstructs user prompts on production Intel TDX. In this paper, we mitigate this channel and provide efficient tokenizer-tailored solution with OTRO. Our central observation is that the tokenizer’s vocabulary and merge tables are read-only during inference, which lets us circumvent the periodic rebuild overhead inherent to in SqrtORAM. OTRO removes this from the critical path in three steps. First, OTRO replicates the tables across a pool of independent SqrtORAM instances. Second, an epoch-based rotation runs the rebuild of depleted instances asynchronously in the background, while continuously serving incoming tokenizer requests from an available instance. Third, chunked tokenization interleaves the tokenization of prompt chunks with GPU prefill, overlapping CPU rebuild work with GPU computation to further reduce the number of instances the pool holds. Together, these techniques turn the bursty rebuild cost of SqrtORAM into background work that does not stall the serving pipeline. Our prototype on Llama-3.1, Qwen3, Gemma-3, and Phi-3 keeps TTFT overhead within 4.5% of the unprotected baseline while reducing the observable trace to prompt length alone, demonstrating that workload-aware ORAM integration is a viable path to end-to-end confidentiality in production LLM-serving stacks.

References

- [1] R. Weiss, D. Ayzenshteyn, and Y. Mirsky, “What was your prompt? a remote keylogging attack on {AI} assistants,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 3367–3384.
- [2] J. Hayase, A. Liu, Y. Choi, S. Oh, and N. A. Smith, “Data mixture inference attack: Bpe tokenizers reveal training data compositions,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 8956–8983, 2024.
- [3] Z. Gao, J. Hu, F. Guo, Y. Zhang, Y. Han, S. Liu, H. Li, and Z. Lv, “I know what you said: Unveiling hardware cache side-channels in local large language model inference,” *arXiv preprint arXiv:2505.06738*, 2025.
- [4] T. Zhang, G. Saileshwar, and D. Lie, “Time will tell: Timing side channels via output token count in large language models,” *arXiv preprint arXiv:2412.15431*, 2024.

- [5] N. Carlini, D. Paleka, K. D. Dvijotham, T. Steinke, J. Hayase, A. F. Cooper, K. Lee, M. Jagielski, M. Nasr, A. Conmy, I. Yona, E. Wallace, D. Rolnick, and F. Tramèr, “Stealing part of a production language model,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.06634>
- [6] “Amd secure encrypted virtualization (sev),” <https://www.amd.com/en/developer/sev.html>, AMD, accessed: 2024-10-07.
- [7] A. Sev-Snp, “Strengthening vm isolation with integrity protection and more,” *White Paper*, January, vol. 53, no. 2020, pp. 1450–1465, 2020.
- [8] “Intel trust domain extensions,” <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html>, Intel, accessed: 2024-10-07.
- [9] Intel, “Architecture Specification: Intel Trust Domain Extensions (Intel TDX) Module,” 2020.
- [10] Microsoft, “What is Azure confidential computing?” May 2025, accessed: 2025-09-28. [Online]. Available: <https://learn.microsoft.com/en-us/azure/confidential-computing/overview>
- [11] NVIDIA, “Confidential Compute on NVIDIA Hopper H100,” <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/HCC-Whitepaper-v1.0.pdf>, 2023.
- [12] NVIDIA, “AI security with confidential computing,” <https://www.nvidia.com/en-us/data-center/solutions/confidential-computing/>.
- [13] Meta, “Private processing technical whitepaper,” Meta AI, Tech. Rep., June 2025, <https://ai.meta.com/static-resource/private-processing-technical-whitepaper>.
- [14] Google Cloud, “How Confidential Accelerators can boost AI workload security,” 2025.
- [15] WhatsApp. About Private Processing. [Online]. Available: <https://faq.whatsapp.com/2089630958184255>
- [16] M. Li, Y. Zhang, H. Wang, K. Li, and Y. Cheng, “{CIPHERLEAKS}: Breaking constant-time cryptography on {AMD}-{SEV} via the ciphertext side channel,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 717–732.
- [17] M. Morbitzer, M. Huber, J. Horsch, and S. Wessel, “SEVered: Subverting AMD’s virtual machine encryption,” in *11th European Workshop on Systems Security*. ACM, 2018.
- [18] J. Werner, J. Mason, M. Antonakakis, M. Polychronakis, and F. Monrose, “The SEVerEST of them all: Inference attacks against secure virtual enclaves,” in *ACM Asia Conference on Computer and Communications Security*. ACM, 2019, pp. 73–85.
- [19] Y. Yarom and K. Falkner, “Flush+ reload: A high resolution, low noise, l3 cache side-channel attack,” in *USENIX Security Symposium*, 2014, pp. 719–732.
- [20] F. Dall, G. De Micheli, T. Eisenbarth, D. Genkin, N. Heninger, A. Moghimi, and Y. Yarom, “Cachequote: Efficiently recovering long-term secrets of sgx epid via cache attacks,” 2018.
- [21] J. Götzfried, M. Eckert, S. Schinzel, and T. Müller, “Cache attacks on intel sgx,” in *Proceedings of the 10th European Workshop on Systems Security*, 2017, pp. 1–6.
- [22] J. Van Bulck, F. Piessens, and R. Strackx, “SGX-Step: A practical attack framework for precise enclave execution control,” in *Proceedings of the 2nd Workshop on System Software for Trusted Execution*, 2017, pp. 1–6.
- [23] Y. Yan, W. Huang, I. Grishchenko, G. Saileshwar, A. Mehta, and D. Lie, “Relocate-vote: Using sparsity information to exploit ciphertext side-channels,” in *Proceedings of the 34th USENIX Conference on Security Symposium*, 2025, pp. 5699–5717.
- [24] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, “Hypertheft: Thieving model weights from tee-shielded neural networks via ciphertext side channels,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4346–4360.
- [25] M. Li, L. Wilke, J. Wichelmann, T. Eisenbarth, R. Teodorescu, and Y. Zhang, “A systematic look at ciphertext side channels on amd sev-snp,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 337–351.
- [26] G. Team, “Gemma 3,” 2025. [Online]. Available: <https://goo.gle/Gemma3Report>
- [27] AI@Meta, “Llama 3 model card,” 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [28] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Fragoso, J. Gao, M. Gao, M. Gao, A. Garg, A. D. Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacrose, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, and X. Zhou, “Phi-3 technical report: A highly capable language model locally on your phone,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [29] Q. Team, “Qwen3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [30] T. Zhang, “Time will tell: Timing side channels via output token count,” Master’s thesis, University of Toronto (Canada), 2025.
- [31] T. Hornetz, H. Yavarzadeh, A. Cheu, A. Gascon, L. Gerlach, D. Moghimi, P. Schoppmann, M. Schwarz, and R. Zhang, “Tdxray: Microarchitectural side-channel analysis of intel tdx for real-world workloads.”
- [32] R. Rajat, Y. Wang, and M. Annavaram, “Laoram: A look ahead oram architecture for training large embedding tables,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [33] H. Hashemi, W. Xiong, L. Ke, K. Maeng, M. Annavaram, G. E. Suh, and H.-H. S. Lee, “Data leakage via access patterns of sparse features in deep learning-based recommendation systems,” *arXiv preprint arXiv:2212.06264*, 2022.
- [34] M. Umar, A. P. Marathe, M. D. Gupta, S. J. Ghosh, G. E. Suh, and W. Xiong, “Efficient memory side-channel protection for embedding generation in machine learning,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 423–441.
- [35] O. Goldreich and R. Ostrovsky, “Software protection and simulation on oblivious rams,” *J. ACM*, vol. 43, no. 3, p. 431–473, May 1996. [Online]. Available: <https://doi.org/10.1145/233551.233553>
- [36] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, and S. Devadas, “Path oram: An extremely simple oblivious ram protocol,” in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (CCS)*, 2013.
- [37] L. Ren, C. Fletcher, A. Kwon, E. Stefanov, E. Shi, M. van Dijk, and S. Devadas, “Constants count: Practical improvements to oblivious RAM,” in *24th USENIX Security Symposium*, 2015.
- [38] D. Kaplan, J. Powell, and T. Woller, “AMD memory encryption,” *White paper*, 2016.

- [39] Intel, “Intel Hardware Shield- Intel Total Memory Encryption,” <https://www.intel.com/content/www/us/en/architecture-and-technology/total-memory-encryption-security-paper.html>, 2017.
- [40] —, “Intel trust domain extensions whitepaper,” <https://software.intel.com/content/dam/develop/external/us/en/documents/tdx-whitepaper-final9-17.pdf>, 2020.
- [41] AMD, “SEV API version 0.22,” 2019.
- [42] ARM, “ARM CCA Security Model 1.0,” 2021.
- [43] V. Costan and S. Devadas, “Intel SGX explained,” *IACR Cryptol. ePrint Arch.*, vol. 2016, no. 86, pp. 1–118, 2016.
- [44] AMD, “AMD64 architecture programmer’s manual volume 2: System programming,” 2019.
- [45] D. Kaplan, “Upcoming x86 technologies for malicious hypervisor protection,” https://static.sched.com/hosted_files/lsseu2019/65/SEV-SNP%20Slides%20Nov%201%202019.pdf, 2020.
- [46] —, “Protecting VM register state with SEV-ES,” *White paper*, 2017.
- [47] Nvidia, “NVIDIA Attestation,” <https://docs.nvidia.com/attestation/index.html>, 2025.
- [48] Meta AI, “Private processing for whatsapp: Technical whitepaper,” <https://ai.meta.com/static-resource/private-processing-technical-whitepaper>, Jun. 2025.
- [49] Microsoft, “Azure AI Confidential Inferencing: Technical Deep-Dive,” <https://techcommunity.microsoft.com/blog/azureconfidentialcomputingblog/azure-ai-confidential-inferencing-technical-deep-dive/4253150>, 2025.
- [50] Mithril Security. (2023) Blindai: Confidential ai deployment with trusted execution environments. [Online]. Available: <https://github.com/mithril-security/blindai>
- [51] —. (2024) Blindllama: Zero-trust confidential ai for llms. [Online]. Available: <https://blindllama.mithrilsecurity.io/en/latest/>
- [52] Edgeless Systems. (2024, Jul.) Continuum: The first confidential llm platform to revolutionize secure, privacy-preserving ai with nvidia h100 gpus. [Online]. Available: <https://www.edgeless.systems/blog/launching-confidential-llm-platform-continuum-ai>
- [53] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, 2016, pp. 1715–1725.
- [54] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, Łukasz Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. Corrado, M. Hughes, and J. Dean, “Google’s neural machine translation system: Bridging the gap between human and machine translation,” 2016. [Online]. Available: <https://arxiv.org/abs/1609.08144>
- [55] T. Kudo and J. Richardson, “Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” *arXiv preprint arXiv:1808.06226*, 2018.
- [56] A. Moi and N. Patry, “Huggingface’s tokenizers,” accessed: 2025-11-22. [Online]. Available: <https://github.com/huggingface/tokenizers>
- [57] “Nvidia confidential computing,” <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/HCC-Whitepaper-v1.0.pdf>, NVIDIA, accessed: 2024-10-07.
- [58] A. Seto, O. K. Duran, S. Amer, J. Chuang, S. van Schaik, D. Genkin, and C. Garman, “Wiretap: Breaking server sgx via dram bus interception,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 708–722.
- [59] J. De Meulemeester, L. Wilke, D. Oswald, T. Eisenbarth, I. Verbauwhede, and J. Van Bulck, “Badram: Practical memory aliasing attacks on trusted execution environments,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 4117–4135.
- [60] R. Rajat, Y. Wang, and M. Annavaram, “Pageoram: An efficient dram page aware oram strategy,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 91–107.
- [61] X. Yu, “nano-vllm,” 2025. [Online]. Available: <https://github.com/GeeekExplorer/nano-vllm>
- [62] J. Götzfried, M. Eckert, S. Schinzel, and T. Müller, “Cache attacks on intel sgx,” in *Proceedings of the 10th European Workshop on Systems Security*, 2017, pp. 1–6.
- [63] M. Lipp, A. Kogler, D. Oswald, M. Schwarz, C. Easdon, C. Canella, and D. Gruss, “Platypus: Software-based power side-channel attacks on x86,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 355–371.
- [64] F. Brasser, U. Müller, A. Dmitrienko, K. Kostiaainen, S. Capkun, and A.-R. Sadeghi, “Software grand exposure: {SGX} cache attacks are practical,” in *11th USENIX workshop on offensive technologies (WOOT 17)*, 2017.
- [65] F. Dall, G. De Micheli, T. Eisenbarth, D. Genkin, N. Heninger, A. Moghimi, and Y. Yarom, “Cachequote: Efficiently recovering long-term secrets of sgx epid via cache attacks,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 2, 2018.
- [66] A. Moghimi, G. Irazoqui, and T. Eisenbarth, “Cachezoom: How sgx amplifies the power of cache attacks,” in *International conference on cryptographic hardware and embedded systems*. Springer, 2017, pp. 69–90.
- [67] M. Schwarz, S. Weiser, D. Gruss, C. Maurice, and S. Mangard, “Malware guard extension: Using sgx to conceal cache attacks,” in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2017, pp. 3–24.
- [68] J. Van Bulck, F. Piessens, and R. Strackx, “Nemesis: Studying microarchitectural timing leaks in rudimentary cpu interrupt logic,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 178–195.
- [69] B. Chor, E. Kushilevitz, O. Goldreich, and M. Sudan, “Private information retrieval,” *Journal of the ACM (JACM)*, vol. 45, no. 6, pp. 965–981, 1998.
- [70] E. Kushilevitz and R. Ostrovsky, “Replication is not needed: Single database, computationally-private information retrieval,” in *Proceedings 38th annual symposium on foundations of computer science*. IEEE, 1997, pp. 364–373.
- [71] P. Mishra, R. Poddar, J. Chen, A. Chiesa, and R. A. Popa, “Obliv: An efficient oblivious search index,” in *2018 IEEE symposium on security and privacy (SP)*. IEEE, 2018, pp. 279–296.
- [72] A. Tinoco, S. Gao, and E. Shi, “{EnigMap}:{External-Memory} oblivious map for secure enclaves,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4033–4050.
- [73] L. Ren, X. Yu, C. W. Fletcher, M. Van Dijk, and S. Devadas, “Design space exploration and optimization of path oblivious ram in secure processors,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, 2013, pp. 571–582.
- [74] L. Ren, C. Fletcher, A. Kwon, E. Stefanov, E. Shi, M. Van Dijk, and S. Devadas, “Constants count: Practical improvements to oblivious {RAM},” in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 415–430.
- [75] M. Raoufi, Y. Zhang, and J. Yang, “Ir-oram: Path access type based memory intensity reduction for path-oram,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 360–372.
- [76] M. Raoufi, J. Yang, X. Tang, and Y. Zhang, “Ep-oram: efficient nvm-friendly path eviction for ring oram in hybrid memory,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

- [77] —, “Ab-oram: Constructing adjustable buckets for space reduction in ring oram,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 361–373.
- [78] G. Liu, K. Li, Z. Xiao, and R. Wang, “Ps-oram: Efficient crash consistency support for oblivious ram on nvme,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 188–203.
- [79] W. He, F. Wang, and D. Feng, “H2oram: Low response latency optimized oram for hybrid memory systems,” in *2020 IEEE 38th International Conference on Computer Design (ICCD)*. IEEE, 2020, pp. 405–408.
- [80] Y. Che and R. Wang, “Multi-range supported oblivious ram for efficient block data retrieval,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 369–382.
- [81] D. Cao, M. Zhang, H. Lu, X. Ye, D. Fan, Y. Che, and R. Wang, “Streamline ring oram accesses through spatial and temporal optimization,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 14–25.
- [82] G. Asharov, I. Komargodski, W.-K. Lin, K. Nayak, E. Peserico, and E. Shi, “Oporama: optimal oblivious ram,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2020, pp. 403–432.
- [83] S. Sasy, S. Gorbunov, and C. W. Fletcher, “ZeroTrace: Oblivious memory primitives from intel sgx,” in *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*, 2018. [Online]. Available: <https://www.ndss-symposium.org/ndss2018/zerotrace-oblivious-memory-primitives-intel-sgx/>
- [84] A. Ahmad, K. Kim, M. I. Sarfaraz, and B. Lee, “Obliviate: A data oblivious filesystem for intel sgx,” in *NDSS*, 2018.
- [85] M. Tran, L. Luu, M. S. Kang, I. Bentov, and P. Saxena, “Obscuro: A bitcoin mixer using trusted execution environments,” in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 692–701.
- [86] L. Zheng, Z. Zhang, W. Dong, Y. Zhang, Y. Wu, and C. Wang, “{H2O2RAM}: A {High-Performance} hierarchical doubly oblivious {RAM},” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 8501–8520.
- [87] S. Johnson, V. Scarlata, C. Rozas, E. Brickell, and F. Mckeen, “Intel software guard extensions: Epid provisioning and attestation services,” 2016.
- [88] S. Tople, Y. Jia, and P. Saxena, “Pro-oram: Constant latency read-only oblivious ram,” *Cryptology ePrint Archive*, 2018.
- [89] L. Reichert, G. R. Chandran, P. Schoppmann, T. Schneider, and B. Scheuermann, “Menhir: an oblivious database with protection against access and volume pattern leakage,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1675–1690.
- [90] J. Wichelmann, A. Rabich, A. Pättschke, and T. Eisenbarth, “Obelix: Mitigating side-channels through dynamic obfuscation,” in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 4182–4199.
- [91] K. E. Batchier, “Sorting networks and their applications,” in *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, 1968, pp. 307–314.

Appendix A. Artifact Evaluation

The paper’s core contributions concern (i) obfuscating memory-access patterns of LLM tokenization (vocabulary and merge tables) using ORAM-based mechanisms, and (ii)

evaluating the performance overheads relevant to serving (tokenization latency, tokenization-to-TTFT ratio, and initialization cost).

- **System Configuration** is introduced in the Section 7.1. Additionally, we use CUDA-12.4 with PyTorch 2.6.0, and flash_attn-2.7.4+cu124torch2.6-cp312-cp312-linux_x86_64.whl in the Guest VM.

- **nano-vLLM Modification.**

Link: <https://anonymous.4open.science/r/oram-nanovllm-E3CD/README.md>

Content: Modification of nano-vllm to support Llama-3.1, Gemma-3, and Phi-3. Included ttft.py can be used to obtain TTFT given a benchmark and input length.

- **Tokenizer Implementation**

Link: <https://anonymous.4open.science/r/oram-tokenizer-065D/README.md>

Content: ORAM-protected tokenizer library (installation is same as the original Transformer’s Tokenizer library). Modify tokenizers/src/model/bpe/model.rs to support different configurations of ORAM structures.

Appendix B. Security Analysis

Here we provide the proofs for two lemmas used in the proof sketch of Theorem 1 in Section 6.

Lemma 1 (Per-Access Trace Indistinguishability). *For all $i \in [N]$, the random variables $\mathcal{T}(a_i^1)$ and $\mathcal{T}(a_i^2)$, denoting the physical DRAM access traces induced by the i -th logical access in executions $\text{Exec}(1)$ and $\text{Exec}(2)$, are identically distributed, i.e.,*

$$\mathcal{T}(a_i^1) \equiv_d \mathcal{T}(a_i^2).$$

Proof. We analyze the physical DRAM access trace generated by the i -th logical access and show that it is identically distributed for $\text{Exec}(1)$ or $\text{Exec}(2)$. We decompose the trace into two components: 1) accesses incurred during the ORAM read operation, and 2) accesses incurred during reshuffling.

Read phase. For the ORAM read phase, the controller deterministically selects the SqrtORAM instance to access based solely on the logical access index i . Specifically, the j -th SqrtORAM instance is selected, where

$$j = \left\lfloor \frac{i-1}{\sqrt{N}} \right\rfloor \bmod n_{\text{pool}} + 1$$

and n_{pool} denotes the number of replicated SqrtORAM instances. Since this selection depends only on i and public system parameters, the same SqrtORAM instance is accessed in executions $\text{Exec}(1)$ and $\text{Exec}(2)$. Within the selected SqrtORAM, the physical location accessed is determined by evaluating a pseudorandom function on the logical address. Pseudorandomness of the PRF makes the resulting locations computationally indistinguishable from a uniform random choice over all physical locations in the SqrtORAM.

Consequently, for any physical location ℓ in the SqrtORAM, the probability that ℓ is accessed during the read phase is

$$\Pr[\text{access } \ell] = \frac{1}{N + \sqrt{N}}$$

In addition, the controller performs a linear scan of the stash during every read operation. This scan accesses the same sequence of physical locations regardless of the logical address being accessed, and hence contributes to identical distribution in $\mathcal{T}(a_i^1)$ and $\mathcal{T}(a_i^2)$. Therefore, the distribution of physical DRAM accesses generated during the ORAM read phase of the i -th logical access is identical in executions $\text{Exec}(1)$ and $\text{Exec}(2)$.

Reshuffling. We next consider the accesses generated by reshuffling. The system triggers an oblivious reshuffle of the j -th SqrtORAM instance after $\sqrt{N}$ logical accesses to that instance for fixed public parameters j and N . Since the reshuffle schedule depends only on the logical access index i and public parameters, reshuffles are triggered at the same logical times in executions $\text{Exec}(1)$ and $\text{Exec}(2)$. Moreover, the reshuffling algorithm is oblivious by construction [91], meaning that the sequence of physical DRAM accesses it generates depends only on the size of the data structure and not on its contents or access history. As a result, the distribution of DRAM access traces generated by reshuffling is identical across executions $\text{Exec}(1)$ and $\text{Exec}(2)$.

Combining the above arguments, both the ORAM read accesses and the reshuffling accesses generated by the i -th logical access are identically distributed in executions $\text{Exec}(1)$ and $\text{Exec}(2)$. Therefore, the overall physical DRAM access trace distribution of $\mathcal{T}(a_i^1)$ is identical to that of $\mathcal{T}(a_i^2)$. $\square$

Lemma 2 (Independence of Logical Access Traces). *The physical DRAM access traces generated by distinct logical accesses to the vocab and merge tables are mutually independent random variables.*

Proof. Consider two distinct logical access indices $i \neq i'$, corresponding to logical addresses a_i and $a_{i'}$. We show that the physical DRAM access traces generated by these accesses are independent. We distinguish two cases for the read phase and reshuffling depending on whether the accesses are served by the same SqrtORAM instance.

Read phase Case 1: Accesses served by the same SqrtORAM instance. Suppose that a_i and $a_{i'}$ are mapped to the same SqrtORAM instance. By construction, each logical access invokes PRF to determine the physical access location within the SqrtORAM. The security guarantee of SqrtORAM ensures that the physical access locations corresponding to distinct logical accesses are computationally indistinguishable from independent random locations. Consequently, the physical DRAM access traces generated by accesses a_i and $a_{i'}$ are independent random variables.

Read phase Case 2: Accesses served by different SqrtORAM instances. Next, suppose that a_i and $a_{i'}$ are mapped to different SqrtORAM instances. Each SqrtORAM instance is initialized independently using independent secret keys and randomness. Therefore, the pseudorandom

function evaluations used to select physical access locations in different SqrtORAM instances are independent. Accordingly, the physical DRAM access traces generated by accesses to different SqrtORAM instances are independent.

Reshuffling accesses. Finally, we consider accesses generated by reshuffling. Reshuffling is triggered according to a fixed, public schedule that depends only on the number of logical accesses performed. The reshuffling algorithm is oblivious, and the sequence of physical DRAM accesses it generates depends only on public parameters. Consequently, reshuffling accesses are independent of the logical accesses that trigger them, as well as of reshuffling operations triggered at other times.

Combining the above cases, the physical DRAM access traces generated by any two distinct logical accesses are independent random variables. $\square$